

[Skip to main content](#)[Back to top](#)[scikit-learn homepage](#)[scikit-learn homepage](#)

- [Install](#)
- [User Guide](#)
- [API](#)
- [Examples](#)
- [Community](#)
- [More](#)
 - [Getting Started](#)
 - [Release History](#)
 - [Glossary](#)
 - [Development](#)
 - [FAQ](#)
 - [Support](#)
 - [Related Projects](#)
 - [Roadmap](#)
 - [Governance](#)
 - [Institutional support](#)
 - [About us](#)



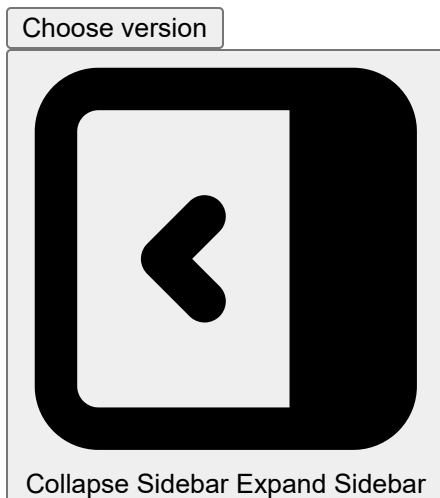
- [System Settings](#)
- [Light](#)
- [Dark](#)
- [GitHub](#)

[Choose version](#)

- [Install](#)
- [User Guide](#)
- [API](#)
- [Examples](#)
- [Community](#)
- [Getting Started](#)
- [Release History](#)
- [Glossary](#)
- [Development](#)
- [FAQ](#)
- [Support](#)
- [Related Projects](#)
- [Roadmap](#)
- [Governance](#)
- [Institutional support](#)
- [About us](#)



- [System Settings](#)
- [Light](#)
- [Dark](#)
- [GitHub](#)



Section Navigation

- [sklearn](#)
 - ▶
- [sklearn.base](#)
 - ▶
- [sklearn.calibration](#)
 - ▶
- [sklearn.callback](#)
 - ▶
- [sklearn.cluster](#)
 - ▶
- [sklearn.compose](#)
 - ▶
- [sklearn.covariance](#)
 - ▶
- [sklearn.cross_decomposition](#)
 - ▶
- [sklearn.datasets](#)
 - ▶
- [sklearn.decomposition](#)
 - ▶
- [sklearn.discriminant_analysis](#)
 - ▶
- [sklearn.dummy](#)
 - ▶
- [sklearn.ensemble](#)
 - ▶
- [sklearn.exceptions](#)
 - ▶
- [sklearn.experimental](#)

- ▶
- [sklearn.feature_extraction](#)
- ▶
- [sklearn.feature_selection](#)
- ▶
- [sklearn.frozen](#)
- ▶
- [sklearn.gaussian_process](#)
- ▶
- [sklearn.impute](#)
- ▶
- [sklearn.inspection](#)
- ▶
- [sklearn.isotonic](#)
- ▶
- [sklearn.kernel_approximation](#)
- ▶
- [sklearn.kernel_ridge](#)
- ▶
- [sklearn.linear_model](#)
- ▶
- [sklearn.manifold](#)
- ▶
- [sklearn.metrics](#)
 - ▼
 - [check_scoring](#)
 - [get_scorer](#)
 - [get_scorer_names](#)
 - [make_scorer](#)
 - [accuracy_score](#)
 - [auc](#)
 - [average_precision_score](#)
 - [balanced_accuracy_score](#)
 - [brier_score_loss](#)
 - [class_likelihood_ratios](#)
 - [classification_report](#)
 - [cohen_kappa_score](#)
 - [confusion_matrix](#)
 - [confusion_matrix_at_thresholds](#)
 - [d2_brier_score](#)
 - [d2_log_loss_score](#)
 - [deg_score](#)
 - [det_curve](#)
 - [f1_score](#)
 - [fbeta_score](#)
 - [hamming_loss](#)
 - [hinge_loss](#)
 - [jaccard_score](#)
 - [log_loss](#)
 - [matthews_corrcoef](#)
 - [metric_at_thresholds](#)
 - [multilabel_confusion_matrix](#)
 - [ndcg_score](#)
 - [precision_recall_curve](#)

- [precision_recall_fscore_support](#)
- [precision_score](#)
- [recall_score](#)
- [roc_auc_score](#)
- [roc_curve](#)
- [top_k_accuracy_score](#)
- [zero_one_loss](#)
- [d2_absolute_error_score](#)
- [d2_pinball_score](#)
- [d2_tweedie_score](#)
- [explained_variance_score](#)
- [max_error](#)
- [mean_absolute_error](#)
- [mean_absolute_percentage_error](#)
- [mean_gamma_deviance](#)
- [mean_pinball_loss](#)
- [mean_poisson_deviance](#)
- [mean_squared_error](#)
- [mean_squared_log_error](#)
- [mean_tweedie_deviance](#)
- [median_absolute_error](#)
- [r2_score](#)
- [root_mean_squared_error](#)
- [root_mean_squared_log_error](#)
- [coverage_error](#)
- [label_ranking_average_precision_score](#)
- [label_ranking_loss](#)
- [adjusted_mutual_info_score](#)
- [adjusted_rand_score](#)
- [calinski_harabasz_score](#)
- [contingency_matrix](#)
- [pair_confusion_matrix](#)
- [completeness_score](#)
- [davies_bouldin_score](#)
- [fowlkes_mallows_score](#)
- [homogeneity_completeness_v_measure](#)
- [homogeneity_score](#)
- [mutual_info_score](#)
- [normalized_mutual_info_score](#)
- [rand_score](#)
- [silhouette_samples](#)
- [silhouette_score](#)
- [v_measure_score](#)
- [consensus_score](#)
- [DistanceMetric](#)
- [additive_chi2_kernel](#)
- [chi2_kernel](#)
- [cosine_distances](#)
- [cosine_similarity](#)
- [distance_metrics](#)
- [euclidean_distances](#)
- [haversine_distances](#)
- [kernel_metrics](#)
- [laplacian_kernel](#)

- [linear_kernel](#)
- [manhattan_distances](#)
- [nan_euclidean_distances](#)
- [paired_cosine_distances](#)
- [paired_distances](#)
- [paired_euclidean_distances](#)
- [paired_manhattan_distances](#)
- [pairwise_kernels](#)
- [polynomial_kernel](#)
- [rbf_kernel](#)
- [sigmoid_kernel](#)
- [pairwise_distances](#)
- [pairwise_distances_argmin](#)
- [pairwise_distances_argmin_min](#)
- [pairwise_distances_chunked](#)
- [ConfusionMatrixDisplay](#)
- [DetCurveDisplay](#)
- [PrecisionRecallDisplay](#)
- [PredictionErrorDisplay](#)
- [RocCurveDisplay](#)
- [sklearn.mixture](#)
 - ▶
- [sklearn.model_selection](#)
 - ▶
- [sklearn.multiclass](#)
 - ▶
- [sklearn.multioutput](#)
 - ▶
- [sklearn.naive_bayes](#)
 - ▶
- [sklearn.neighbors](#)
 - ▶
- [sklearn.neural_network](#)
 - ▶
- [sklearn.pipeline](#)
 - ▶
- [sklearn.preprocessing](#)
 - ▶
- [sklearn.random_projection](#)
 - ▶
- [sklearn.semi_supervised](#)
 - ▶
- [sklearn.svm](#)
 - ▶
- [sklearn.tree](#)
 - ▶
- [sklearn.utils](#)
 - ▶
-
- [API Reference](#)
- [sklearn.metrics](#)
- [r2_score](#)

r2_score#

`sklearn.metrics.r2_score(y_true, y_pred, *, sample_weight=None, multioutput='uniform_average', force_finite=True)`[\[source\]](#)#

R^2 (coefficient of determination) regression score function.

Best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse). In the general case when the true y is non-constant, a constant model that always predicts the average y disregarding the input features would get a R^2 score of 0.0.

In the particular case when y_{true} is constant, the R^2 score is not finite: it is either NaN (perfect predictions) or -Inf (imperfect predictions). To prevent such non-finite numbers to pollute higher-level experiments such as a grid search cross-validation, by default these cases are replaced with 1.0 (perfect predictions) or 0.0 (imperfect predictions) respectively. You can set `force_finite` to False to prevent this fix from happening.

Note: when the prediction residuals have zero mean, the R^2 score is identical to the [Explained Variance score](#).

Read more in the [User Guide](#).

Parameters:

y_truearray-like of shape (n_samples,) or (n_samples, n_outputs)

Ground truth (correct) target values.

y_predarray-like of shape (n_samples,) or (n_samples, n_outputs)

Estimated target values.

sample_weightarray-like of shape (n_samples,), default=None

Sample weights.

multioutput{‘raw_values’, ‘uniform_average’, ‘variance_weighted’}, array-like of shape (n_outputs,) or None, default=‘uniform_average’

Defines aggregating of multiple output scores. Array-like value defines weights used to average scores. Default is “uniform_average”.

‘raw_values’ :

Returns a full set of scores in case of multioutput input.

‘uniform_average’ :

Scores of all outputs are averaged with uniform weight.

‘variance_weighted’ :

Scores of all outputs are averaged, weighted by the variances of each individual output.

Changed in version 0.19: Default value of multioutput is ‘uniform_average’.

force_finitebool, default=True

Flag indicating if NaN and -Inf scores resulting from constant data should be replaced with real numbers (1.0 if prediction is perfect, 0.0 otherwise). Default is True, a convenient setting for hyperparameters' search procedures (e.g. grid search cross-validation).

Added in version 1.1.

Returns:

zfloat or ndarray of floats

The R^2 score or ndarray of scores if 'multioutput' is 'raw_values'.

Notes

This is not a symmetric function.

Unlike most other scores, R^2 score may be negative (it need not actually be the square of a quantity R).

This metric is not well-defined for single samples and will return a NaN value if n_samples is less than two.

References

[1]

[Wikipedia entry on the Coefficient of determination](#)

Examples

```
>>> from sklearn.metrics import r2_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> r2_score(y_true, y_pred)
0.948...
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> r2_score(y_true, y_pred,
...         multioutput='variance_weighted')
0.938...
>>> y_true = [1, 2, 3]
>>> y_pred = [1, 2, 3]
>>> r2_score(y_true, y_pred)
1.0
>>> y_true = [1, 2, 3]
>>> y_pred = [2, 2, 2]
>>> r2_score(y_true, y_pred)
0.0
>>> y_true = [1, 2, 3]
>>> y_pred = [3, 2, 1]
>>> r2_score(y_true, y_pred)
-3.0
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2]
>>> r2_score(y_true, y_pred)
1.0
>>> r2_score(y_true, y_pred, force_finite=False)
nan
>>> y_true = [-2, -2, -2]
>>> y_pred = [-2, -2, -2 + 1e-8]
>>> r2_score(y_true, y_pred)
```

```
0.0
>>> r2_score(y_true, y_pred, force_finite=False)
-inf
```

Gallery examples#

[Effect of transforming the targets in regression model](#)

Effect of transforming the targets in regression model

[Failure of Machine Learning to infer causal effects](#)

Failure of Machine Learning to infer causal effects

[L1-based models for Sparse Signals](#)

L1-based models for Sparse Signals

[Non-negative least squares](#)

Non-negative least squares

[Ordinary Least Squares and Ridge Regression](#)

Ordinary Least Squares and Ridge Regression

[previous](#)

[median_absolute_error](#)

[next](#)

[root_mean_squared_error](#)

On this page

- [r2_score](#)
- [Gallery examples](#)

scikit-learn is [financially supported](#) by Probabl and other institutions.

 [Probabl](#)

[Enterprise-grade solutions and services](#)

© Copyright 2007 - 2026, scikit-learn developers (BSD License).